



DESARROLLO DE SOFTWARE PARA PLATAFORMAS MOVILES

JONATHAN LÓPEZ LONDOÑO
JLOPEZL@UAO.EDU.CO
315 926 5443



PWA

They are web applications that use modern technologies to offer a native-like experience.

They are accessed through a web browser but can be installed on the user's device and function offline, using technologies such as Service Workers and Web App Manifest.



PWA

Advantages

- ✓ Cross-Platform
- ✓ Automatic Updates
- ✓ Offline Access
- ✓ Easy Installation

Disadvantages

- ✓ Limited Access to Device Features
- ✓ Performance
- ✓ Availability in App Stores

Creating a PWA

We will follow 5 steps to transform a web platform into a pwa:

1. Create the Manifest
2. Connect it to my web.
3. Create the service-worker.js
4. Register into main.jsx
5. Publish in HTTPS

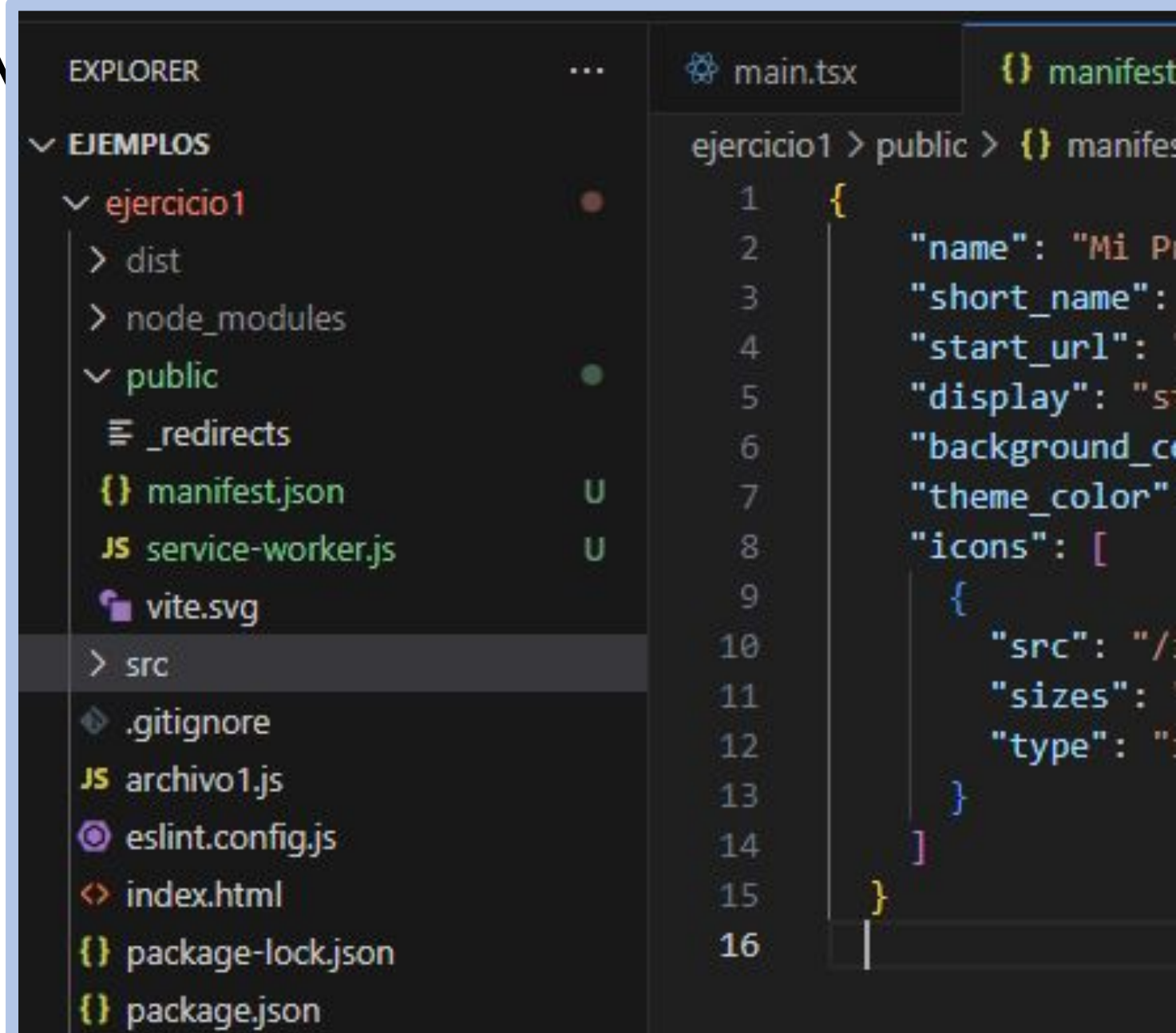


Creating a PWA

Creating the manifest.json

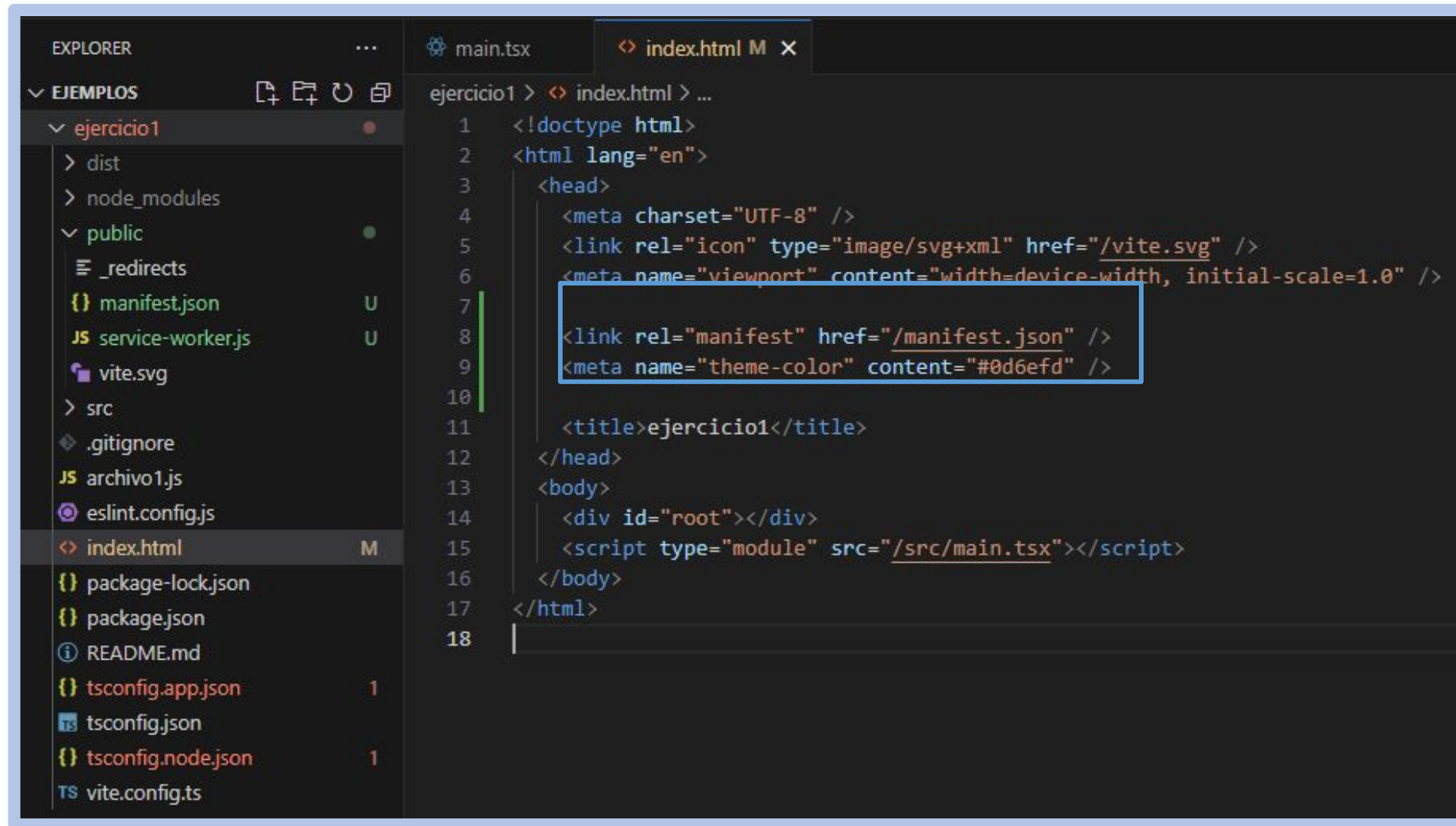
Define:

- ✓ The name of the app
- ✓ Its icon
- ✓ Its color
- ✓ How it should be opened



Connecting the manifest.json

We need to connect the manifest in the index.html



The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left and the Editor on the right. The Explorer sidebar shows a project structure with a folder named 'ejercicio1' containing files like 'dist', 'node_modules', 'public', '_redirects', 'manifest.json', 'service-worker.js', 'vite.svg', 'src', '.gitignore', 'archivo1.js', 'eslint.config.js', 'index.html', 'package-lock.json', 'package.json', 'README.md', 'tsconfig.app.json', 'tsconfig.json', 'tsconfig.node.json', and 'vite.config.ts'. The 'index.html' file is selected and highlighted. The Editor shows the content of 'index.html' with the following code:

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7
8     <link rel="manifest" href="/manifest.json" />
9     <meta name="theme-color" content="#0d6efd" />
10
11   <title>ejercicio1</title>
12 </head>
13 <body>
14   <div id="root"></div>
15   <script type="module" src="/src/main.tsx"></script>
16 </body>
17 </html>
18
```

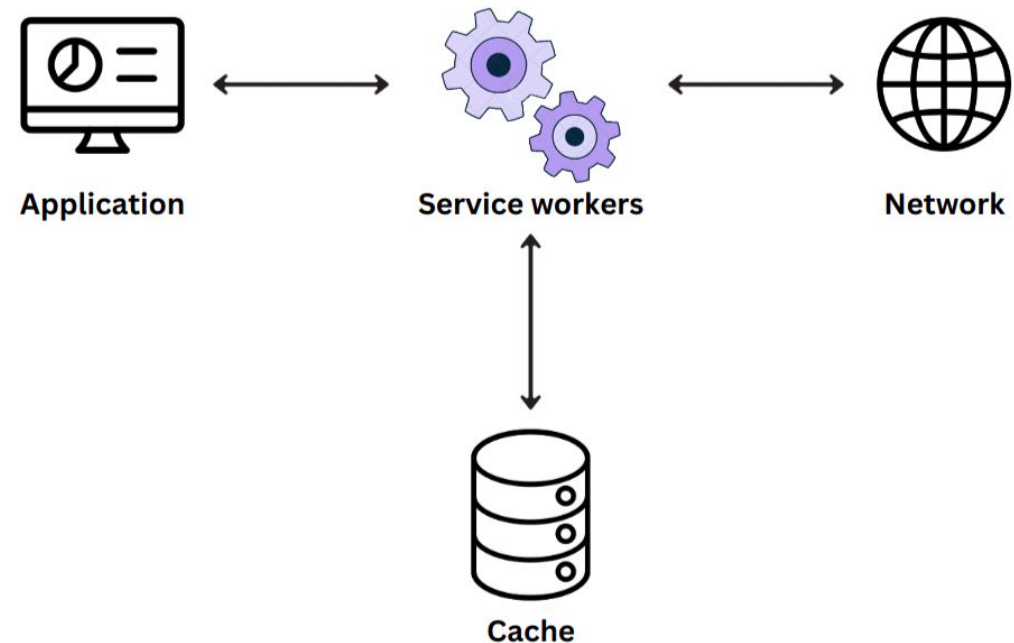
The line `<link rel="manifest" href="/manifest.json" />` is highlighted with a blue box, indicating the connection to the manifest file.

Creating a PWA

Creating the service-worker.js

A Service Worker is a special JavaScript file that:

- ✓ runs in the background
- ✓ is independent of the page
- ✓ can intercept network requests
- ✓ can cache data



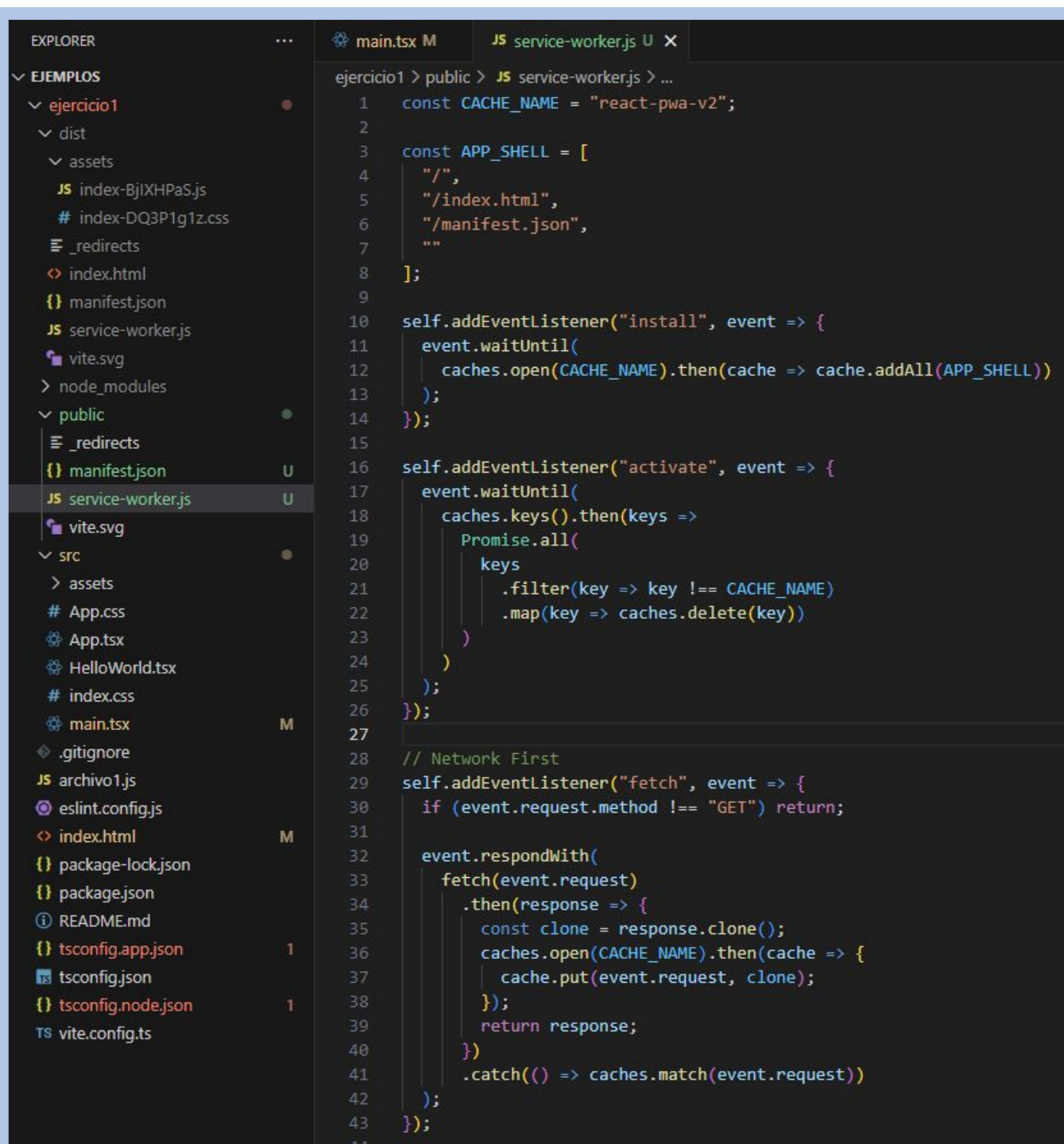
Creating the service-worker.js

We need to create the service worker into the public folder.

For this example, the service worker will be created as **Network First**.

We can configure it as:

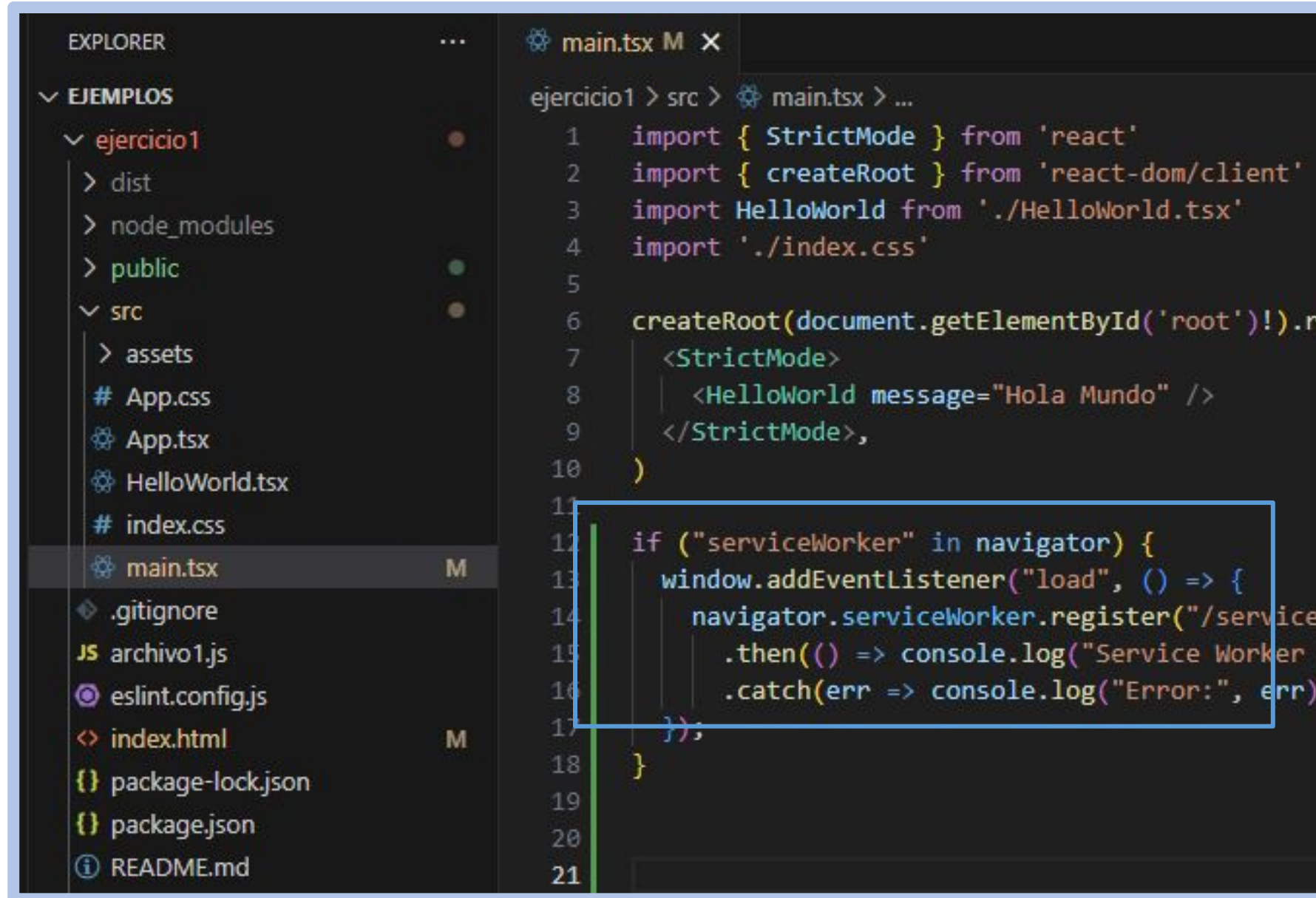
- ✓ Cache First
- ✓ Network First
- ✓ Stale While Revalidate
- ✓ Network only
- ✓ Cache only



```
main.tsx M JS service-worker.js U X
ejercicio1 > public > JS service-worker.js > ...
1  const CACHE_NAME = "react-pwa-v2";
2
3  const APP_SHELL = [
4    "/",
5    "/index.html",
6    "/manifest.json",
7    ""
8  ];
9
10 self.addEventListener("install", event => {
11   event.waitUntil(
12     caches.open(CACHE_NAME).then(cache => cache.addAll(APP_SHELL))
13   );
14 });
15
16 self.addEventListener("activate", event => {
17   event.waitUntil(
18     caches.keys().then(keys =>
19       Promise.all(
20         keys
21           .filter(key => key !== CACHE_NAME)
22           .map(key => caches.delete(key))
23       )
24     )
25   );
26 });
27
28 // Network First
29 self.addEventListener("fetch", event => {
30   if (event.request.method !== "GET") return;
31
32   event.respondWith(
33     fetch(event.request)
34       .then(response => {
35         const clone = response.clone();
36         caches.open(CACHE_NAME).then(cache => {
37           cache.put(event.request, clone);
38         });
39       })
40       .catch(() => caches.match(event.request))
41   );
42 });
43
```


Registering the service-worker.js

We need to register the service worker in the main.tsx



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays a file tree for a project named 'EJEMPLOS'. The 'src' directory is expanded, showing files like 'assets', 'App.css', 'App.tsx', 'HelloWorld.tsx', 'index.css', and 'main.tsx'. The 'main.tsx' file is selected and highlighted. On the right, the main editor displays the content of 'main.tsx'. The code includes imports for 'StrictMode' from 'react', 'createRoot' from 'react-dom/client', 'HelloWorld' from './HelloWorld.tsx', and './index.css'. It then uses 'createRoot' to render the 'HelloWorld' component. A blue box highlights a new code block added at the bottom of the file, which checks for the 'serviceWorker' in the 'navigator' and registers it if present. The code is as follows:

```
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import HelloWorld from './HelloWorld.tsx'
4  import './index.css'
5
6  createRoot(document.getElementById('root')!).render(
7    <StrictMode>
8      <HelloWorld message="Hola Mundo" />
9    </StrictMode>,
10  )
11
12  if ("serviceWorker" in navigator) {
13    window.addEventListener("load", () => {
14      navigator.serviceWorker.register("/service-worker.js")
15        .then(() => console.log("Service Worker registered"))
16        .catch(err => console.log("Error:", err))
17    });
18  }
```

Service Worker Hybrid Strategy

Resource	Strategy	Why
HTML	Network First	To avoid old versions
JS / CSS	Cache First	Because of the HASH
Images	Cache First - Stale While Revalidate	Performance
API's	Network First	Updated Data

Challenge 02

Based on the React app from the Challenge 01:

- Add one image to your parent component
- Transform it into a PWA using Hybrid Strategy
- Add a custom icon
- Deploy the PWA in Netlify
- Add the Netlify Link to the Readme.md and describe how to install the app in the cellphone

